Linear Attention: A Simple Introduction

Matthew Zhang

Myrtle.ai

Outline [1]

1. Recap: Softmax Attention
2. Linear Attention
3. Evolution of Linear Attention Models
4. Code: Linear Attention v.s. Transformer

[1] This presentation is mainly based on: Katharopoulos et al. (2020), *Transformers are RNNs.*

Softmax Attention is $\mathcal{O}(T^2)$

Let's begin with our favorite *softmax attention*.

$$V' = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right) V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{T \times D}$

Reminder: Matrix Multiplication Complexity

For the multiplication of two matrices $A \in \mathbb{R}^{M \times N}$ and $B \in \mathbb{R}^{N \times P}$, the time complexity of computing $C = AB$ is $\mathcal{O}(M \cdot N \cdot P)$.

Operation	**Shape Change**	**Complexity**
AB	$(M, N) \times (N, P) \rightarrow (M, P)$	$\mathcal{O}(MNP)$

- Each of the $M \times P$ elements in the resulting matrix requires N scalar multiplications and additions.

Softmax Attention is $\mathcal{O}(T^2)$

$$V' = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{T \times D}$

- **Training complexity**

Operation	**Shape Change**	**Complexity**
QK^T	$(T, D) \times (D, T) \to (T, T)$	$\mathcal{O}(T^2 D)$
$\text{softmax}(\cdot)$	$(T, T) \to (T, T)$	$\mathcal{O}(T^2)$
$(\cdot)V$	$(T, T) \times (T, D) \to (T, D)$	$\mathcal{O}(T^2 D)$

- **Non associativity** With softmax, we cannot swap the order of matrix multiplication, i.e.,

$$\text{softmax}(QK^T)V \neq Q(K^T V)$$

What if? Restoring Associativity

- **The hypothesis:** If we can decompose $\text{softmax}(QK^T)$ into $\Phi(Q)\Phi(K)^T$, for $\Phi(Q) \in \mathbb{R}^{T \times D'}$, we can leverage the **associative property** of matrix multiplication.

$$V' = \underbrace{\Phi(Q)}_{\in \mathbb{R}^{T \times D'}} \left(\underbrace{\Phi(K)^T}_{\in \mathbb{R}^{D' \times T}} \underbrace{V}_{\in \mathbb{R}^{T \times D}} \right) \tag{2}$$

- **New training complexity:**

Operation	**Shape Change**	**Complexity**
$\Phi(K)^T V$	$(D', T) \times (T, D) \to (D', D)$	$\mathcal{O}(TDD')$
$\Phi(Q)(\cdot)$	$(T, D') \times (D', D) \to (T, D)$	$\mathcal{O}(TDD')$

Let's Derive Linear Attention

Start with the softmax attention in Equation 1. Consider the i-th query, and unpack the softmax:

$$V_i' = \frac{\sum_{j=1}^{T} \mathrm{sim}(Q_i, K_j) V_j}{\sum_{j=1}^{T} \mathrm{sim}(Q_i, K_j)} \tag{3}$$

where $\mathrm{sim}(Q_i, K_j) = \exp\left(\frac{Q_i K_j^\top}{\sqrt{d_k}}\right)$ is called a *kernel* function that measure the similarity between two inputs Q_i and K_j.

Question: Can we find a feature map $\phi(\cdot)$ such that $\mathrm{sim}(Q_i, K_j) = \phi(Q_i)\phi(K_j)$?

Let's Derive Linear Attention

By substituting the linearized similarity $\text{sim}(Q_i, K_j) = \phi(Q_i)\phi(K_j)^\top$ into the attention formula:

$$V_i' = \frac{\sum_{j=1}^{T} \left(\phi(Q_i)\phi(K_j)^\top\right) V_j}{\sum_{j=1}^{T} \phi(Q_i)\phi(K_j)^\top} \tag{4}$$

Since $\phi(Q_i)$ is constant with respect to the summation index j, we apply the distributive property to move it outside the sum:

$$V_i' = \frac{\phi(Q_i)\left(\sum_{j=1}^{T} \phi(K_j)^\top V_j\right)}{\phi(Q_i)\left(\sum_{j=1}^{T} \phi(K_j)^\top\right)} \tag{5}$$

This factorization allows the terms in parentheses to be computed first and avoid the $\mathcal{O}(T^2)$ complexity!

Let's Derive Linear Attention: Matrix Form

To compute attention for all queries simultaneously, we define:

- $\Phi(Q) \in \mathbb{R}^{T \times D'}$
- $\Phi(K) \in \mathbb{R}^{T \times D'}$
- $V \in \mathbb{R}^{T \times D}$
- $\mathbf{1} \in \mathbb{R}^{T \times 1}$ (vector of ones)

This simplifies Equation 5 to

$$V_i' = \frac{\phi(Q_i)(\Phi(K)^\top V)}{\phi(Q_i)\Phi(K)^\top \mathbf{1}} \tag{6}$$

The full linearized attention for the entire sequence is:

$$V' = \frac{\Phi(Q)\left(\Phi(K)^\top V\right)}{\Phi(Q)\left(\Phi(K)^\top \mathbf{1}\right)} \tag{7}$$

Note: The division is performed row-wise.

Let's Derive Linear Attention: Training Complexity

Operation	**Shape Change**	**Complexity**
Numerator: $\Phi(Q)(\Phi(K)^\top V)$		
$\Phi(K)^\top V$	$(D', T) \times (T, D) \to (D', D)$	$\mathcal{O}(TDD')$
$\Phi(Q)(\cdot)$	$(T, D') \times (D', D) \to (T, D)$	$\mathcal{O}(TDD')$
Denominator: $\Phi(Q)(\Phi(K)^\top \mathbf{1})$		
$\Phi(K)^\top \mathbf{1}$	$(D', T) \times (T, 1) \to (D', 1)$	$\mathcal{O}(TD')$
$\Phi(Q)(\cdot)$	$(T, D') \times (D', 1) \to (T, 1)$	$\mathcal{O}(TD')$
Normalization		
Row-wise Division	(T, D) by $(T, 1)$	$\mathcal{O}(TD)$

- The total training complexity is dominated by $\mathcal{O}(TDD')$.
- **Result:** We have successfully replaced $\mathcal{O}(T^2)$ with linear $\mathcal{O}(T)$.

Deriving the Feature Map: 1st Order Approx

To linearize attention, we seek a feature map $\phi(\cdot)$ such that $\mathrm{sim}(Q_i, K_j) \approx \phi(Q_i)\phi(K_j)^\top$.

Consider the first-order Taylor expansion of the exponential (ignoring the $\sqrt{d_k}$ scaling):

$$\exp(Q_i K_j^\top) \approx 1 + Q_i K_j^\top \tag{8}$$

For row vectors $Q_i, K_j \in \mathbb{R}^{1\times d}$, we define the feature map:

$$\phi(x) = \begin{bmatrix} 1 & x \end{bmatrix} \in \mathbb{R}^{1\times(1+d)} \tag{9}$$

Then the inner product $\phi(Q_i)\phi(K_j)^\top$ is:

$$\begin{bmatrix} 1 & Q_i \end{bmatrix} \begin{bmatrix} 1 \\ K_j^\top \end{bmatrix} = 1 + Q_i K_j^\top \tag{10}$$

This shows that a simple concatenation of a bias term and the raw input recovers the first-order approximation of the exponential kernel.

Deriving the Feature Map: Full Exponential Kernel

To recover the exact softmax attention, we expand the exponential to its full Taylor series:

$$\exp(Q_i K_j^\top) = \sum_{n=0}^{\infty} \frac{(Q_i K_j^\top)^n}{n!} = 1 + Q_i K_j^\top + \frac{(Q_i K_j^\top)^2}{2!} + \dots \tag{11}$$

This corresponds to an **infinite-dimensional** feature map $\phi(x)$. To ensure the inner product $\phi(Q_i)\phi(K_j)^\top$ distributes the $1/n!$ correctly, we define:

$$\phi(x) = \left[1, x, \frac{x^{\otimes 2}}{\sqrt{2!}}, \dots, \frac{x^{\otimes n}}{\sqrt{n!}}, \dots\right] \tag{12}$$

where $x^{\otimes n}$ is the n-th order self-tensor product.

What is the $\phi(\cdot)$ used in literature?

In practice, any feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{D'}$ defines a valid *kernel* if it satisfies the **positivity constraint**:

$$\phi(x) \geq 0 \quad \text{(element-wise)} \tag{13}$$

This ensures numerical stability and prevents a zero or negative denominator.

Model	**Feature Map** $\phi(x)$	**Goal**
Linear Transformer	$\text{elu}(x) + 1$	Efficiency/Simplicity
Performer	Random Fourier Features	Softmax Approx.
Mamba-2 (SSD)	Implicit (State Expansion)	

Causal Masking in Linear Attention

To train autoregressive models, position i must only be influenced by positions $j \leq i$. The linearized formulation becomes:

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)} \tag{14}$$

By defining the cumulative states S_i and Z_i:

$$S_i = \sum_{j=1}^{i} \phi(K_j) V_j^T, \quad Z_i = \sum_{j=1}^{i} \phi(K_j) \tag{15}$$

The output is simplified to:

$$V_i' = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i} \tag{16}$$

Note: S_i and Z_i can be updated from S_{i-1} and Z_{i-1} in constant time.

Make Inference Like an RNN

Causal linear attention layers can be viewed as RNNs where S_i and Z_i act as internal hidden states (memory).

State Updates:

$$s_i = s_{i-1} + \phi(x_i W_K)(x_i W_V)^T \tag{17}$$

$$z_i = z_{i-1} + \phi(x_i W_K) \tag{18}$$

Output Prediction:

$$y_i = f_l \left(\frac{\phi(x_i W_Q)^T s_i}{\phi(x_i W_Q)^T z_i} + x_i \right) \tag{19}$$

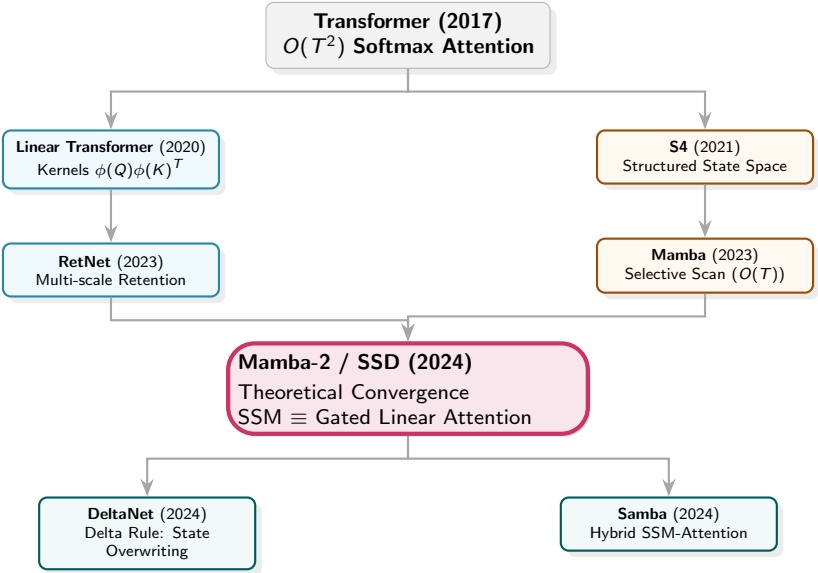
- s_i: Attention memory (feature-value interactions)
- z_i: Normalizer memory (cumulative features)
- f_l: feed-forward + RMSNorm
- x_i: residual connection.

Linear Attention: Summary

$$\text{Attention}(Q, K, V) = \frac{\Phi(Q)\left(\Phi(K)^\top V\right)}{\Phi(Q)\left(\Phi(K)^\top \mathbf{1}\right)} \tag{20}$$

- $\Phi(Q), \Phi(K) \in \mathbb{R}^{T \times D}, \quad V \in \mathbb{R}^{T \times D}$
- Training complexity: $\mathcal{O}(T)$:
- Inference complexity: $\mathcal{O}(1)$
  1. Update KV State: $S_{new} = S_{old} + k_{new}^T v_{new}$ $\mathcal{O}(D^2)$
  2. Update Normalizer: $z_{new} = z_{old} + k_{new}$ $\mathcal{O}(D)$
  3. Compute Output: $y_{new} = f_l\left(\frac{q_{new} S_{new}}{q_{new} z_{new}^T} + x_{new}\right)$ $\mathcal{O}(D^2)$

(Simplified) Evolution of Linear Attention & SSMs



My Coding Demo: Linear Models vs. Llama

Demo Repository:

`https://github.com/MatthewZhang473/linear-attention-demo`

What we are NOT covering today (but I also find interesting)

- Hardware (GPU) efficient algorithms, e.g. block-matrix decomposition
- (Unfortunately) the model architectures

Next: State Space Models

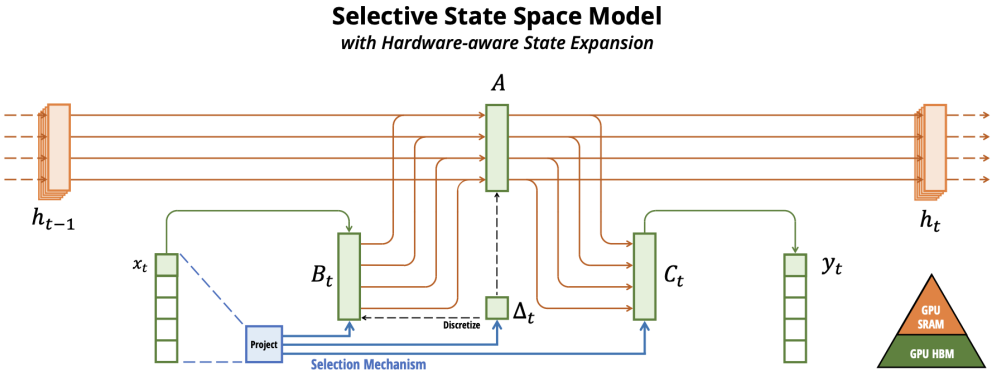


Figure 1: (**Overview.**) Structured SSMs independently map each channel (e.g. $D = 5$) of an input x to output y through a higher dimensional latent state h (e.g. $N = 4$). Prior SSMs avoid materializing this large effective state (DN, times batch size B and sequence length L) through clever alternate computation paths requiring time-invariance: the $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$ parameters are constant across time. Our selection mechanism adds back input-dependent dynamics, which also requires a careful hardware-aware algorithm to only materialize the expanded states in more efficient levels of the GPU memory hierarchy.

References I

1. Vaswani, A. *et al.* Attention is all you need. *Advances in neural information processing systems* **30** (2017).
2. Katharopoulos, A., Vyas, A., Pappas, N. & Fleuret, F. *Transformers are rnns: Fast autoregressive transformers with linear attention.* in *International conference on machine learning* (2020), 5156–5165.
3. Gu, A. & Dao, T. *Mamba: Linear-time sequence modeling with selective state spaces.* in *First conference on language modeling* (2024).
4. Dao, T. & Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).
5. Sun, Y. *et al.* Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621* (2023).
6. Yang, S., Kautz, J. & Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464* (2024).

References II

7. Choromanski, K. *et al.* Rethinking attention with performers. *arXiv preprint arXiv:2009.14794* (2020).